Study Notes Manager — Complete Frontend Implementation Plan

1. Project Objective

Build a complete frontend for Study Notes Manager using:

- **Next.js App Router** for pages, layouts, and server-side functionality.
- **TypeScript** for type safety.
- **Axios** for API requests.
- **Zod** for request and response validation.
- **Auth.js (NextAuth)** for login sessions.
- **FastAPI** as the backend API.
- **LangGraph** for AI-based quiz generation.
- **Server-Sent Events (SSE)** for streaming quiz-generation progress.
- **Tailwind CSS** for styling.

The frontend should support authentication, notes CRUD, AI quiz generation, profile management, and OpenAI API-key management.

The frontend must use real backend data. Remove hardcoded dashboard statistics, fake login, fake registration, simulated API-key verification, and local-only CRUD operations.

Phase 1 — Inspect and Stabilize the Backend

Step 1.1: Understand the existing API

The repository currently exposes these main routes.

Method	Endpoint	Purpose
POST	`/users/login`	Authenticate and receive JWT
POST	`/users/`	Register a user
GET	`/users/`	Get all users; protected

Method	Endpoint	Purpose
GET	`/users/{user_id}`	Current implementation has incorrect response behavior
PUT	`/users/{user_id}`	Update user
DELETE	`/users/{user_id}`	Delete user
GET	`/notes/{user_id}`	Get a user's notes
POST	`/notes/create_note`	Create a note
PUT	`/notes/update_note/{note_id}`	Update a note
DELETE	`/notes/delete_note/{note_id}`	Delete a note
POST	`/notes/quiz_generation/{note_id}`	Generate a quiz using SSE
POST	`/openai/set-key`	Set an API key for the current request context
GET	`/openai/validate-key`	Validate key availability in the current request context

Verify these routes in FastAPI's `/docs` before integrating them.

Step 1.2: Fix backend authentication and authorization

Do this before connecting the frontend.

A. Add a proper current-user endpoint

Implement `GET /users/me`, protected by `get_current_user`, returning the authenticated user's `user_id` and `user_name`.

This lets Auth.js identify the user after receiving the JWT from `/users/login`.

B. Fix user ownership checks

Every protected route must verify that the authenticated user owns the requested resource.

For example, a user must not be able to read, edit, delete, or generate a quiz for another user's notes simply by changing a URL or ID.

Apply this to note CRUD, user updates, user deletion, and quiz generation.

C. Fix `GET /users/{user_id}`

The existing handler checks for a note and returns a note, even though its declared response model is `UserResponse`. It should return a user, or be replaced with a proper `GET /users/me` endpoint.

D. Fix the user update schema

The current backend `UserUpdate` schema requires a password even when the user only wants to change their username. Make update fields optional as appropriate and validate them.

E. Fix OpenAI key handling

The current `ContextVar` approach does not persist a key between separate HTTP requests. Calling `/openai/set-key` will not reliably make that key available to a later quiz-generation request.

Use a deliberate design, such as a short-lived server-side session store or an encrypted, appropriately scoped key-management mechanism. Do not store user API keys in browser local storage or log them.

F. Handle empty notes correctly

`GET /notes/{user_id}` currently returns `404` when a user has no notes. Prefer `200 OK` with an empty array `[]`, so the frontend can display an empty state without treating it as an error.

G. Review database consistency

Make sure note creation derives ownership from the authenticated user rather than trusting a client-provided `user_id`. Add suitable database relationships and cascading behavior where appropriate.

Do not begin production frontend integration until the API's authentication and authorization behavior is correct.

Phase 2 — Install Frontend Dependencies

Step 2.1: Install packages

Run these commands inside the `frontend` directory:

```
pnpm add axios next-auth zod
pnpm add -D vitest @testing-library/react @testing-library/jest-dom
```

Zod is already present in the project, so the command may simply report it as up to date.

Step 2.2: Configure environment variables

Create `frontend/.env.local`:

```
API_BASE_URL=http://127.0.0.1:8000
AUTH_SECRET=replace-with-a-random-secret
AUTH_URL=http://localhost:3000
```

Generate a random secret with:

```
python -c "import secrets; print(secrets.token_hex(32))"
```

Keep `.env.local` out of Git. Never use `NEXT_PUBLIC_` for the backend URL if it is intended to remain server-only.

Phase 3 — Organize the Frontend

Keep the existing UI components when possible. Add or reorganize code into the following structure:

```
frontend/
├── app/
│   ├── (auth)/
│   │   ├── login/page.tsx
│   │   └── register/page.tsx
│   ├── (app)/
│   │   └── dashboard/
│   │       ├── layout.tsx
│   │       ├── page.tsx
│   │       ├── notes/
│   │       │   ├── page.tsx
│   │       │   ├── create/page.tsx
│   │       │   └── [noteId]/
```

```
│   │   │   │   ├── page.tsx
│   │   │   │   └── edit/page.tsx
│   │   │   ├── quiz/page.tsx
│   │   │   └── profile/page.tsx
│   │   └── api/
│   │       ├── auth/[...nextauth]/route.ts
│   │       ├── register/route.ts
│   │       └── backend/[...path]/route.ts
├── components/
│   ├── layout/
│   ├── notes/
│   ├── quiz/
│   └── ui/
├── lib/
│   ├── api/
│   │   ├── axios.ts
│   │   ├── notes.ts
│   │   ├── users.ts
│   │   └── openai.ts
│   ├── schemas/
│   │   ├── auth.ts
│   │   ├── note.ts
│   │   ├── user.ts
│   │   └── quiz.ts
│   └── utils/
├── auth.ts
├── types/
│   ├── index.ts
│   └── next-auth.d.ts
└── .env.local
```

The route groups `(auth)` and `(app)` do not appear in the browser URL. Use a single, consistent URL structure such as `/dashboard/notes` and `/dashboard/profile`.

Phase 4 — Set Up Zod Schemas

Step 4.1: Reuse existing schemas

The project already defines several schemas in `types/index.ts`, including:

- `NoteCreateSchema`
- `NoteUpdateSchema`

- `NoteResponseSchema`
- `UserCreateSchema`
- `UserUpdateSchema`
- `UserResponseSchema`
- `TokenSchema`
- `QuizGenerationResponseSchema`
- `OpenAIKeyRequestSchema`
- `OpenAIKeyResponseSchema`

Move schemas into dedicated files gradually, or keep the current file temporarily and import from it. Avoid defining the same schema twice.

Step 4.2: Separate form input from API payload

For example, a note creation form should collect only:

```
{
  title: string;
  subject: string;
  content: string;
}
```

The backend should derive `user_id` from the JWT and generate or validate `note_id`.

Do not trust the browser to choose the owner of a note.

Step 4.3: Validate responses

After an API response arrives, validate it:

```
const result = NoteResponseSchema.safeParse(response.data);

if (!result.success) {
  throw new Error("The server returned an unexpected note response.");
}

return result.data;
```

Use `safeParse` for user-controlled input and API boundaries. Handle validation errors with useful messages rather than rendering unexpected data.

Phase 5 — Implement Authentication with Auth.js

Step 5.1: Connect login to FastAPI

FastAPI's `/users/login` expects form-encoded credentials through `OAuth2PasswordRequestForm`, not a JSON body.

The login process must:

1. Validate the login form using Zod.
2. Submit `username` and `password` as `application/x-www-form-urlencoded`.
3. Receive `access_token` and `token_type`.
4. Validate the token response with `TokenSchema`.
5. Call `GET /users/me` using `Authorization: Bearer <token>`.
6. Validate the returned user.
7. Store the backend access token in the server-side Auth.js JWT session.
8. Redirect the user to `/dashboard`.

Do not return the backend access token as a browser-visible session property.

Step 5.2: Create the Auth.js configuration

Create `auth.ts` at the frontend project root. Configure the Credentials provider and JWT session strategy.

The `authorize` function should call FastAPI, validate the responses, and return a minimal user object containing the user's ID and username.

Use the session callback to expose safe user information only. Keep the backend access token in the server-side JWT.

Create `app/api/auth/[...nextauth]/route.ts` to export the Auth.js handlers.

Step 5.3: Implement registration

Registration should:

1. Validate username and password using Zod.
2. Call `POST /users/` with JSON.
3. Handle `400` errors such as a duplicate username.

4. Validate the response using `UserResponseSchema`.
5. Redirect to login or automatically sign in after successful registration.

Never log passwords or store them in the browser.

Step 5.4: Protect dashboard pages

Use Auth.js `auth()` in server layouts or pages to protect authenticated areas. Redirect unauthenticated users to `/login`.

Protect backend requests independently too. A protected frontend page alone is not an authorization mechanism.

Step 5.5: Implement logout

Use Auth.js `signOut()` to clear the application session and redirect to `/login`.

Phase 6 — Build a Secure FastAPI Proxy

Step 6.1: Why a proxy is useful

The browser should call Next.js endpoints rather than receive or manage the backend JWT directly.

The flow becomes:

```
Browser
   ↓
Next.js frontend
   ↓
Next.js server route
   ↓  Authorization: Bearer <backend JWT>
FastAPI
   ↓
Database / LangGraph
```

Step 6.2: Proxy requirements

Create `app/api/backend/[...path]/route.ts`.

The proxy should:

- Verify the Auth.js session.
- Read the backend JWT on the server.
- Allow only explicitly approved backend paths and HTTP methods.
- Forward the authorization header server-side.
- Validate or constrain request bodies where appropriate.
- Return backend status codes and useful error messages.
- Avoid forwarding cookies or arbitrary incoming headers to FastAPI.
- Set `cache: "no-store"` for authenticated data.
- Avoid exposing the backend JWT in responses or logs.

Do not create an unrestricted proxy that forwards any URL or method. Do not rely only on a frontend-supplied user ID to enforce ownership.

Phase 7 — Configure Axios

Create `lib/api/axios.ts`:

```
import axios from "axios";

export const api = axios.create({
  baseURL: "/api/backend",
  timeout: 30_000,
  headers: {
    "Content-Type": "application/json",
  },
});
```

This Axios instance calls the Next.js proxy, not FastAPI directly.

Create separate service files for each domain. Keep request logic out of React components.

For example, `lib/api/notes.ts` should contain functions such as:

- `getNotes(userId)`
- `createNote(input)`
- `updateNote(noteId, input)`
- `deleteNote(noteId)`

The service layer should validate the response with the appropriate Zod schema.

For login, registration, and Auth.js callbacks, use the server-side functions appropriate to that flow rather than exposing backend credentials to browser code.

Phase 8 — Implement Notes CRUD

Step 8.1: Notes listing

Replace the hardcoded `INITIAL_NOTES` array in the notes page.

The page should:

1. Obtain the authenticated user's ID.
2. Fetch notes from `GET /notes/{user_id}`.
3. Validate the returned array.
4. Display loading, error, and empty states.
5. Search and filter the actual fetched notes.
6. Implement sorting using the selected sort state.
7. Navigate to a note's detail page when selected.

Step 8.2: Create note

Replace the console-only submit handler in the create-note page.

On submit:

1. Validate the form using Zod.
2. Disable the submit button while saving.
3. Call `POST /notes/create_note`.
4. Display API validation or server errors.
5. On success, redirect to the note detail page or notes list.
6. Refresh or invalidate relevant data.

Do not hardcode a username such as `alexm`. Use the authenticated user's data.

Step 8.3: Note detail page

Implement `app/(app)/dashboard/notes/[noteId]/page.tsx`.

The current repository is missing a proper note-detail page. Add a backend endpoint to retrieve a single note by ID, enforcing ownership, or use the existing user's notes list and find the note within that validated response.

Display the title, subject, content, creation date, and available quiz information.

Step 8.4: Update note

Use `PUT /notes/update_note/{note_id}`.

Validate the editable fields, show saving state, handle errors, and update the UI after success.

Step 8.5: Delete note

Use `DELETE /notes/delete_note/{note_id}`.

Ask for confirmation, call the API, and remove the note from the UI only after the backend confirms success.

Phase 9 — Make the Dashboard Dynamic

Replace all hardcoded values in the dashboard.

The dashboard should use real notes and calculate values such as:

- Total notes
- Notes grouped by subject
- Recently updated notes
- Recent activity based on available timestamps
- Quiz completion or generation information, if the backend stores it

If the backend does not provide a metric, either calculate it from available data or add a backend endpoint. Do not display fabricated statistics.

Create a dashboard service or endpoint if fetching every note just to calculate several statistics becomes inefficient.

Phase 10 — Implement Quiz Generation and SSE

Step 10.1: Start quiz generation

Use:

`POST /notes/quiz_generation/{note_id}`

This endpoint streams server-sent events rather than returning a normal JSON response immediately.

A normal Axios `post()` expecting a complete JSON response is not sufficient for progressive browser streaming.

Step 10.2: Choose the streaming strategy

Use a browser `fetch()` request to a protected Next.js route that streams the upstream response, or use a server-side streaming implementation that safely forwards SSE events.

The Next.js route must authenticate the request, attach the backend JWT, and forward the response stream without buffering the entire result.

Step 10.3: Parse SSE events

Handle the backend's event names:

- `workflow_update`
- `complete`
- `error`

The UI should display:

1. Starting generation
2. Current workflow step
3. Completion state
4. Final generated questions and any supported telemetry
5. An actionable error message if generation fails

Cancel the reader or abort the request when the user leaves the page or cancels the operation.

Do not treat a successful HTTP status as proof that the AI workflow completed successfully. Check for the `complete` event.

Phase 11 — Implement OpenAI API-Key Management

The existing API-key modal currently simulates verification.

Replace that behavior with a real backend integration, but first fix the backend's request-scoped key handling.

Requirements:

- Validate key input using Zod.
- Never print or log the key.
- Never put the key in local storage, URLs, analytics, or browser-visible session data.
- Display only a safe confirmation after successful verification.
- Make it clear whether the key is saved, session-scoped, or required again.
- Do not assume `POST /openai/set-key` persists the key across requests.

A key-management design must be completed before quiz generation can reliably use a key provided through the UI.

Phase 12 — Implement Profile Management

Replace hardcoded profile values and simulated updates.

Profile page

- Fetch the current user through `GET /users/me`.
- Display the authenticated username.
- Populate forms from real user data.
- Show loading and error states.

Username or password updates

- Use Zod validation.
- Call the corresponding backend endpoint.
- Verify ownership in FastAPI.
- Handle duplicate usernames and invalid data.
- If a username change invalidates the current JWT identity, define how the session will be refreshed or reauthenticated.

Account deletion

- Require explicit confirmation.
- Delete only the authenticated user's account.
- Ensure related notes are handled according to the database's intended deletion policy.
- Clear the Auth.js session after successful deletion.
- Redirect to registration or login.

Fix inconsistent navigation links, including any links that point to `/profile` instead of `/dashboard/profile`.

Phase 13 — Improve UI States and Error Handling

Every data-driven page should support the following states:

- Loading
- Empty data
- Successful result
- Validation error
- Unauthorized session
- Forbidden resource
- Resource not found
- Backend unavailable
- Unexpected response

Use reusable components such as `LoadingState`, `EmptyState`, `ErrorMessage`, `ConfirmDialog`, and `SubmitButton`.

Use Next.js `loading.tsx`, `error.tsx`, and `not-found.tsx` where appropriate.

Avoid displaying raw stack traces or internal backend exception details to users.

Phase 14 — Testing Strategy

Authentication tests

- Valid login
- Incorrect password
- Unknown username
- Registration with a duplicate username
- Invalid registration data
- Session persistence after refresh
- Logout
- Expired or invalid backend JWT

Notes tests

- List notes for the current user
- Empty notes list
- Create a note
- Edit a note
- Delete a note
- Open a nonexistent note
- Attempt to access another user's note
- Handle backend `401`, `403`, `404`, and `500` responses

Quiz tests

- Generate a quiz successfully
- Display intermediate workflow updates
- Handle the `complete` event
- Handle the `error` event
- Abort an in-progress request
- Handle a missing or invalid OpenAI key

Quality checks

Run these commands from the frontend directory:

```
pnpm lint
pnpm exec tsc --noEmit
```

```
pnpm build
```

Add tests for service functions, Zod schemas, and important user flows. Test the frontend against the actual FastAPI server rather than relying only on mocked responses.

Phase 15 — Recommended Implementation Order

Follow this order to avoid building dependent features on unstable foundations.

1. Inspect and fix FastAPI authentication and ownership checks.
2. Verify all API routes and response schemas in `/docs`.
3. Install Axios and Auth.js.
4. Implement Zod schemas for login and registration.
5. Connect Auth.js Credentials login to FastAPI.
6. Implement registration.
7. Protect dashboard pages.
8. Implement the authenticated Next.js backend proxy.
9. Create the Axios service layer.
10. Implement notes listing and creation.
11. Implement note detail, editing, and deletion.
12. Replace dashboard mock data.
13. Implement profile management.
14. Fix API-key lifecycle and management.
15. Implement streamed quiz generation.
16. Add loading, empty, and error states.
17. Run tests, lint, TypeScript, and production build.
18. Test cross-user access and token handling.
19. Verify deployment environment variables and HTTPS behavior.

Definition of Done

The frontend is complete when:

- Login and registration use the real backend.
- Authenticated pages are protected.
- Backend JWTs remain server-side.
- Notes CRUD persists in the database.
- Dashboard statistics reflect real data.
- Quiz generation streams progress and displays the final result.

- Profile and API-key functionality use real backend behavior.
- Every feature handles failures and empty states.
- Authorization prevents users from accessing other users' data.
- The frontend passes lint, type-checking, tests, and production build.